

Servey - Website Context

Single-source context primer for the servey.in marketing site. Read this first. It is written so that a person or a model can pick up the work without re-deriving anything from the codebase, the git history, or a previous conversation.

- **Repo:** `github.com/JaneshKapoor/servey-website`
 - **Production:** <https://servey.in> (Vercel, auto-deploys on push to `main`)
 - **Last reviewed:** 31 August 2026
 - **Source of truth:** `docs/CONTEXT.md` . `docs/context.pdf` is generated from it by `npm run context:pdf` - never edit the PDF directly. (That command renders every `docs/*.md` , so new documents are picked up automatically.)
 - **Companion:** `SEO-CONTEXT.md` - the search and answer-engine playbook: what we do, what we deliberately do not do, the query cluster map, and the evidence behind each decision. Read it before changing anything that touches ranking.
-

1. Read this before changing anything

These are the standing rules for the project. They are not style preferences; each exists because breaking it would make the site dishonest, leak private data, or create legal exposure.

#	Rule	Why it exists
1	Never fabricate a product screenshot.	Every image is either a real capture or a styled placeholder in a device frame. <code>components/device-frame.tsx</code> carries the note "Placeholder fill - styled, never a fake product screenshot."
2	No invented ratings, testimonials, download counts, user numbers, or <code>AggregateRating</code> schema.	Servey is pre-launch with zero public users. Any such claim is fabricated, and <code>AggregateRating</code> without real reviews is a Google structured-data violation.
3	Never overclaim in a comparison. Every competitor comparison must state where the competitor is genuinely better.	It is the site's main credibility asset and the reason the comparison posts are citable.
4	Never imply Termius shipped malware.	Termius was the <i>victim</i> of the ZuRu impersonation campaign. Asserting otherwise would be false and defamatory.
5	Do not reveal the app's internal tech stack in external-facing copy.	Product positioning decision. (Describing the <i>website's</i> stack, as this document does, is fine.)
6	Do not solicit upvotes on Product Hunt, AlternativeTo, or anywhere else.	Against their rules; triggers rank penalties.
7	Pricing is pre-launch. Offers use <code>availability: PreOrder</code> . Cards drive to the waitlist, never to checkout.	Nobody can be charged yet.
8	If you add anything that collects data, update <code>app/privacy/page.tsx</code> in the same change.	The policy makes specific promises. Shipping a tracker without updating it makes the published policy false.

Known open issue - carried deliberately

`public/screenshots/hero-devices.png` is a real capture that exposes personal filenames (`Drivinglicensetest.pdf` , `Payment Reciept.pdf` , `GoogleService-Info.plist`), a folder named "Janesh", the host name "Janesh's Mac mini", and port 8765. **It is live on the homepage.** This was raised and the owner's decision was to leave it. Re-shooting it is the fix if that decision ever changes.

2. What Servey is

A native Apple app that puts your Mac in your pocket.

- **Host:** a Mac. **Clients:** iPhone and iPad.
- Full screen mirroring, real mouse/keyboard/trackpad input, and a **real terminal** (a genuine shell on the Mac, not a web console).
- **Two paths, chosen automatically:** hardware-accelerated HEVC on the local network for sharpness; private peer-to-peer end-to-end-encrypted WebRTC when the devices are on different networks. No VPN, no port forwarding.
- Sign in with Google on both devices; Servey only ever pairs devices on the **same account**.
- Native Swift/SwiftUI - deliberately not an Electron or Java port.

Status: pre-launch, waitlist only. There is no downloadable build. This constraint drives most of the growth decisions in §9.

Pricing (lib/content.ts → pricing)

Plan	India	International	Includes
Free	₹0	\$0	Everything, capped at 5-minute sessions, 3 a day , reset at local midnight. No card.
Terminal	₹99/mo	\$1.99/mo	Unlimited terminal time; persistent named sessions; reattach from any device
Full access (featured)	₹299/mo	\$4.49/mo	Everything above + screen mirroring, mouse/keyboard/trackpad, adaptive quality

Monthly subscription, cancel anytime, **not charged until launch**. The free tier is enforced client-side by `SessionLimitManager` in the iOS app (`freeSessionSeconds = 5 * 60` , `freeSessionsPerDay = 3`) - what a paid plan buys is **time, not a longer feature list**, which is why the pricing copy says so explicitly.

System requirements: macOS 15.3+ on the host, iOS/iPadOS 18.5+ on the client. A genuinely narrow window that excludes some visitors - stated on the page deliberately, because finding out after joining a waitlist is worse than finding out before.

Brand

- Tagline: **"Your Mac. In your pocket."**
- Accent: `#22dc6e`
- Contact: `hello@servey.in`
- Socials in `Organization.sameAs` : `x.com/KapoorJanesh` , `x.com/dwivediishivam`

3. Stack

- **Next.js 16.2.10** (App Router) + **React 19.2.4** + **TypeScript**
- **React Compiler** enabled (`reactCompiler: true`)
- **Tailwind CSS v4** using `@theme inline` - tokens live in `app/globals.css`
- **Framer Motion** for reveals and micro-interactions
- **Radix UI** primitives (Dialog, Accordion) behind shadcn-style wrappers
- **lucide-react** icons, **sonner** toasts
- **next/font** - Inter + JetBrains Mono, self-hosted
- **firebase-admin** for waitlist/contact persistence
- **posthog-js** for analytics (lazy-loaded - see §8)

Commands

```
npm run dev          # http://localhost:3000
npm run build         # production build
npm run lint          # eslint
npm run indexnow      # ping IndexNow with the current sitemap URLs
npm run context:pdf   # regenerate docs/context.pdf from docs/CONTEXT.md
npm run seo:audit     # assert the SEO invariants against live (or --base=localhost)
npm run seo:meta      # duplicate + over-length titles/descriptions in .next HTML
npm run seo:framing   # Servey depth / substance / density across every post
npm run ai:files      # regenerate llms.txt, llms-full.txt, ai.txt
```

`ai:files` also runs as `prebuild`, so a build can never ship an `llms.txt` that has drifted from `lib/blog.ts`. `seo:meta` and `seo:framing` read the rendered `.next` output, so run `npm run build` first.

`.github/workflows/checks.yml` runs `tsc`, `lint`, `build`, `seo:meta` and `seo:framing` on every push and PR to `main`. `seo:framing` is `continue-on-error` - it reports against a backlog of pre-existing failures (see SEO-CONTEXT §11) rather than blocking.

4. Directory map

```
app/
  layout.tsx          # fonts, metadata, JSON-LD @graph, theme bootstrap,
                      #   <Analytics/>, Toaster, skip link
  page.tsx            # landing page – composes the sections
  globals.css         # design tokens (dark + light), utilities, keyframes
  [useCase]/page.tsx  # the 7 use-case landing pages (see §5)
  blog/page.tsx       # blog index – H1 "Mac remote access guides"
  blog/[slug]/page.tsx # post renderer + Article/FAQPage/BreadcrumbList JSON-LD
  mac/page.tsx        # the Mac host page – hardcoded in the iOS app's
                      #   onboarding, so it must never move or 404
  privacy/  terms/    # legal pages
  api/waitlist/route.ts # POST – honeypot + rate limit + provider
  api/contact/route.ts  # POST – Firestore `contacts`
  feed.xml/route.ts     # RSS 2.0, force-static
  not-found.tsx        # 404, noindex + follow
  opengraph-image.tsx  # generated 1200x630 OG image
  sitemap.ts  robots.ts icon.png  apple-icon.png
components/
  sections/           # header, hero, features, statement, comparison,
                      #   how-it-works, pricing, faq, trust-strip, footer
  ui/                 # button, input, textarea, select, badge, accordion, dialog
  motion/             # reveal, tilt, magnetic (all honour reduced motion)
  analytics.tsx       # PostHog pageview tracker (Suspense-wrapped)
  device-frame.tsx    # iPad/iPhone/Mac frames + placeholder fills
  waitlist-form.tsx   contact-form.tsx  waitlist-dialog.tsx  contact-dialog.tsx
  dual-path-diagram.tsx  privacy-illustration.tsx  theme-toggle.tsx  wordmark.tsx
lib/
  site.ts             # site constants – name, url, tagline, description, nav
  content.ts          # homepage copy: features, steps, comparison, faqs, pricing
  blog.ts             # all 23 posts + FAQ map + contentUpdated
  use-cases.ts        # the 7 landing pages
  screenshots.ts      # typed registry of image slots
  analytics.ts        # lazy PostHog loader + capture helpers
  waitlist-providers.ts # swappable Firebase / Formspree / Resend / console
  firebase-admin.ts  rate-limit.ts  countries.ts  utils.ts
public/
  brand/              # app-icon logos
  screenshots/        # real captures
  llms.txt            # GENERATED – map for AI answer engines
  llms-full.txt       # GENERATED – every post in full, ~174KB
  ai.txt              # GENERATED – usage terms for AI crawlers
  f79fa191....txt    # IndexNow key file
docs/
  CONTEXT.md          # this document (source of truth)
```

```
SEO-CONTEXT.md      # search + answer-engine playbook (do / do not / why)
context.pdf         # generated – do not edit
seo-context.pdf     # generated – do not edit
benchmark-post-draft.md # unpublished draft; every figure is <<FILL: ...>>
scripts/
  indexnow.mjs      # IndexNow submission
  build-context-pdf.mjs # CONTEXT.md -> context.pdf via headless Chrome
  seo-audit.mjs     # asserts the SEO invariants; exits non-zero on failure
  meta-audit.mjs    # duplicate/over-length titles + descriptions
  servey-depth.mjs  # Servey framing metrics (depth, substance, density)
  build-ai-files.mjs # generates public/llms.txt, llms-full.txt, ai.txt
  benchmark/        # latency + sharpness harness (see its README)
.github/workflows/
  checks.yml        # tsc, lint, build, seo:meta, seo:framing on push/PR
source-material/    # original brief, WebRTC explainer, one-pager
```

5. Content system

All copy is data, not JSX. Three registries drive nearly every page.

`lib/site.ts`

Site-wide constants. `site.url` is the canonical origin - change it here and sitemap, robots, JSON-LD and every canonical tag follow.

`lib/content.ts`

Homepage sections: `trustItems`, `features` (7), `steps` (3), `comparison` (9 rows), `faqs` (10), `pricing` (3 plans × 2 regions). `features` also feeds `SoftwareApplication.featureList` and `faqs` feeds the homepage `FAQPage` - so editing copy here updates the structured data.

`lib/blog.ts` - 23 posts

Blocks are a discriminated union:

```
type Block =
  | { type: "p"; text: string }
  | { type: "h2"; text: string }
  | { type: "ul"; items: string[] }
  | { type: "table"; caption?: string; headers: string[]; rows: string[][] }
  | { type: "img"; src: string; alt: string; width: number; height: number; caption?: string };
```

`table` exists because comparison tables are the structure answer engines extract most reliably. Head-to-head posts should lead with one.

`contentUpdated` (currently `2026-08-16`) is a single date that drives `lastModified` across the sitemap. **Bump it when content is meaningfully revised** - it is deliberately not a build timestamp, because claiming every page changed on every deploy gets the signal discounted.

Slug	Angle
control-your-mac-from-iphone-ipad	Core how-to
screens-jump-desktop-alternative-mac	Alternative-to
screens-vs-jump-desktop	Head-to-head (competitor vs competitor)
access-your-mac-remotely-over-cellular	Cellular / strict networks
control-a-headless-mac-mini-remotely	merged into headless-mac-mini-setup , 301 redirect
real-terminal-on-your-mac-from-iphone	Terminal
run-ai-agents-on-your-mac-remotely	AI agents
who-is-servey-for-developers-home-labs	Audience
run-ai-agents-locally-on-your-mac	Local AI
stay-in-control-of-ai-agents-from-anywhere	AI oversight
best-remote-desktop-for-mac	Honest roundup (9 min)
termius-alternative-mac-terminal	Termius comparison (8 min)
screens-5-alternatives	Roundup - targets screens 5 alternative
jump-desktop-vs-teamviewer	Head-to-head - the proven format, TeamViewer volume
jump-desktop-vs-rustdesk	Head-to-head - free/open-source branch of intent

The last three were added on 16 Aug 2026 in direct response to the query data in §7: competitor-vs-competitor is the only non-brand format earning clicks. In all three, **Servey is deliberately not the hero** - it appears once, late, labelled as our own pre-launch project, and each mention tells the reader to buy an established product if they need something today. That restraint is the reason these pages get read and cited; do not "improve" it into a sales pitch.

Internal linking works by keyword overlap, not anchors. `p` blocks render as plain text - `app/blog/[slug]/page.tsx` parses no markdown or HTML - so the only real internal-link mechanism is exact-string keyword overlap feeding `relatedPosts()`. Engineering the `keywords` array is the link graph.

lib/use-cases.ts - 7 landing pages, served by app/[useCase]/page.tsx

control-mac-from-iphone · terminal-on-iphone · terminal-on-ipad · headless-mac-mini · mac-for-developers · remote-mac-for-ai-agents · mac-home-lab

control-mac-from-iphone was added on 16 Aug 2026 and is **the commercial pillar for the core product intent**, which §7 shows was fragmented across ten phrasings earning zero clicks. It is first in the array, so it leads the cross-link mesh. Its `relatedSlug` points at `control-your-mac-from-iphone-ipad`, which is the *informational* spoke for the same intent - the two must reinforce each other, never compete. If you add another page targeting "control Mac from iPhone", you are re-creating the exact fragmentation this page was built to fix.

Why these exist. Competitor research found Termius uses a landing-page *template technique*: ~70% shared structure, ~30% task-specific content, plus a cross-link mesh between the variants. Servey copies the technique but on a different axis - **task x device** rather than Termius's **SSH client x platform** - because competing on their axis means fighting where they hold #1. Termius does not rank at all for "run terminal on mac from iphone"; the pages that do are small indie apps.

Each entry: `metaTitle` (≤51 chars), `description` (110–160), `h1` , `lede` , 4 tiles, sections, FAQs, `keywords` , `relatedSlug` (**must be a real blog slug**) and `navLabel` for the cross-link mesh.

Routing trap: `[useCase]` is a *top-level* dynamic segment, so it would otherwise swallow every unknown path. `generateStaticParams` + `dynamicParams = false` is what makes unknown paths 404 correctly. Do not remove that export.

6. URL inventory

40 URLs in `sitemap.xml` (29 Sep 2026), priority-ordered:

Priority	URLs
1.0	<code>/</code>
0.9	the 7 use-case pages (<i>above blog - these are the commercial pages</i>) and <code>/mac</code>
0.8	<code>/blog</code>
0.7	the 28 blog posts
0.3	<code>/privacy</code> , <code>/terms</code>

The sitemap also carries **image entries**: the homepage lists every ready screenshot, `/mac` lists the host-app shot, and each post lists only the images it actually embeds.

Not in the sitemap but live: `/robots.txt` , `/sitemap.xml` , `/feed.xml` , `/llms.txt` , `/llms-full.txt` , `/ai.txt` , `/opengraph-image` , `/api/waitlist` , `/api/contact` , `/ingest/*` (analytics proxy), and the 404. The three AI files are **generated at build time** by `scripts/build-ai-files.mjs` - edit `lib/blog.ts` or `lib/content.ts` , never the files in `public/` .

7. SEO system

This is the most invested-in part of the site. Change it carefully.

Structured data

- **Root layout** emits one `@graph : Organization → WebSite → SoftwareApplication` (with `featureList` and two `PreOrder Offers`) → `FAQPage`. Nodes carry stable `@id`s (`#organization` , `#website` , `#app` , `#faq`).
- **Blog posts** add `Article` + `FAQPage` + `Person` + `BreadcrumbList`.
- **Use-case pages** add `WebPage` + `FAQPage` + `BreadcrumbList`, where `WebPage.about` points at `${site.url}/#app` by **@id reference** so the product description can never drift from the layout's copy.

Breadcrumb rule: the number of `ListItem`s in `BreadcrumbList` must equal the number of visible breadcrumb links. A mismatch is exactly what Google penalises. This was shipped wrong once (3 in JSON-LD, 2 visible) and fixed.

FAQ rich results are dead - the markup is not. Google added the deprecation notice on **7 May 2026**: FAQ rich results no longer render, the Search Console report and Rich Results Test support were removed in June 2026, and the API data went in August 2026. **Do not strip the `FAQPage` JSON-LD.** `FAQPage` remains a valid Schema.org type, Google confirms unused structured data does not affect Search, and answer engines still parse it. What changed is the *expectation*: FAQ blocks no longer win SERP real estate, so they must earn their place through user value and AI citability instead. Deprecated alongside the June 2025 batch: `HowTo`, `SpecialAnnouncement`, `ClaimReview`, `VehicleListing`, `EstimatedSalary`, `LearningVideo`, Course carousel. `QAPage` is retained.

Metadata rules

- Title template is `%s – Servey`, which adds **9 characters**. Keep `metaTitle` **≤51 chars** so the rendered title stays ≤60.
- Descriptions **110–160 characters**.
- **Every page sets an explicit `alternates.canonical` and an explicit `openGraph.url`**. Without the latter, OG inherits the homepage URL from the root layout and disagrees with the canonical.
- The canonicals are now doubly load-bearing - see the trailing-slash note in §8.

Crawlers

`app/robots.ts` explicitly allows the AI answer engines by name - `GPTBot`, `OAI-SearchBot`, `ChatGPT-User`, `ClaudeBot`, `Claude-Web`, `Claude-SearchBot`, `PerplexityBot`, `Perplexity-User`, `Google-Extended`, `Applebot-Extended`, `Bingbot`, `Amazonbot`, `CCBot` - because some stacks block them by default and the intent here is to be citable.

IndexNow

`npm run indexnow` submits the sitemap URLs; the key file lives at the domain root. **This matters because ChatGPT Search reads Bing's index**, so an IndexNow ping is the fastest path into an answer engine. Last

submission was accepted (HTTP 200).

`public/llms.txt` , `llms-full.txt` , `ai.txt`

All three are **generated** by `scripts/build-ai-files.mjs` , which runs as `prebuild` . `llms.txt` is the curated map (including "What Servey is used for", pointing at the use-case URLs), `llms-full.txt` carries every post in full (~174KB), and `ai.txt` states the usage terms for AI crawlers. **Do not edit them - the next build overwrites the file.** Change `lib/blog.ts` , `lib/use-cases.ts` or `lib/content.ts` and run `npm run ai:files` .

The generator exists because the hand-written `llms.txt` drifted badly: it was missing 12 posts and still carried the "barely touches our servers" overclaim a month after that line had been removed from every other file on the site.

Current Google Search Console picture (3-month view, to 14 Aug 2026)

16 clicks · 626 impressions · 2.6% CTR · average position 15.8 across 39 queries. Previous reading (to ~2 Aug): 14 clicks · 502 impressions · 2.8% · position 14.9. Impressions are growing; position drifted slightly *down*, which is expected when new pages enter the index low and dilute the average.

Diagnosis: position 15.8 is page two, which takes roughly 1% of clicks versus ~25% on page one. A 2.6% CTR is *above* par for that position, so the titles and descriptions are working - **this is a ranking problem, not a copywriting problem.** The fix is authority and coverage, not more rewrites.

What the query data proves (the most useful thing in this document)

Brand (`servey`) is 7 clicks / 131 impressions - **44% of all clicks from 21% of impressions.** Everything below is the non-brand picture.

Cluster	Impressions	Clicks	Read
Competitor vs competitor (screens vs jump desktop + variants)	~58	2	✅ The only non-brand cluster earning clicks
AI on Mac (run local ai on mac, mac ai agent...)	~15	0	Biggest non-competitor cluster
Control Mac from iPhone (10 phrasings)	~12	0	Core product intent - fragmented
Headless Mac mini (7 phrasings)	~9	0	Fragmented
<code>airplayuiagent</code>	4	0	Accidental, off-intent (a macOS process name)

Three conclusions that should drive content decisions:

1. **The comparison format is the only proven winner** - and specifically *competitor-vs-competitor*, where Servey is not the subject. `screens vs jump desktop` earns clicks; nothing else non-brand does. Build more of these.
2. **The core product intent is fragmented across ~10 phrasings, each with 1-2 impressions and zero clicks** ("control mac with iphone", "iphone control mac", "use iphone as mac remote", "access mac from iphone"...). That pattern means Google reads the site as marginally relevant to all of them and

authoritative on none. The fix is a single strong pillar page for the intent, with the existing blog post as its informational spoke - not more pages competing for the same thing.

3. **Pure how-to content is not ranking.** The informational posts generate impressions but no clicks. Comparison and alternative pages do the work.
-

8. Analytics - PostHog

Added August 2026. Key design decisions, all deliberate:

Lazy-loaded. `lib/analytics.ts` dynamically `import()` `s posthog-js` on first use instead of at module scope. The SDK is ~60 KB gzipped; loading it at module scope would inflate First Load JS on every route, including the landing pages whose entire job is to rank. It lands in its own chunk after hydration.

No-op without a key. If `NEXT_PUBLIC_POSTHOG_KEY` is unset, every helper silently does nothing. Local dev therefore never pollutes production numbers, and no call site needs a guard.

Reverse-proxied. `next.config.ts` rewrites `/ingest/*` to PostHog, because `*.i.posthog.com` sits on default content-blocker lists.

- The rewrites are returned as a **plain array**, which puts them in the `afterFiles` phase - evaluated **before** dynamic routes. That is what stops `app/[useCase]` from swallowing `/ingest/*`.
- `skipTrailingSlashRedirect: true` is required because PostHog's ingest paths end in a slash (`/ingest/e/`) and Next's default 308 would break capture. **Trade-off:** `/blog/` no longer redirects to `/blog`, so the explicit canonical tags described in §7 are what keep duplicates out of the index.

Privacy posture (matched word-for-word by `app/privacy/page.tsx`):

- `person_profiles: "identified_only"` and **nothing calls `identify()`** - so no person profile is created and no analytics data is linked to a waitlist email. Keep it that way unless the policy is updated first.
- `respect_dnt: true` - Do Not Track is honoured. On-brand for a product sold on privacy.
- Session replay **off** unless `NEXT_PUBLIC_POSTHOG_SESSION_RECORDING=true`. Turning it on records real browsing and requires a policy update.
- `capture_pageview: false`; pageviews are captured manually in `components/analytics.tsx`, because the SDK's own listener fires once per document load and misses App Router client navigations.

The Suspense boundary in `components/analytics.tsx` is load-bearing. `useSearchParams()` opts a route out of static rendering unless wrapped. Every page here is statically generated, so removing the boundary would turn the whole site dynamic.

Events

Event	Properties	Meaning
<code>\$pageview</code>	<code>\$current_url</code>	Manual, fires on every route change
<code>waitlist_submitted</code>	<code>source</code> , <code>country</code>	Passed validation, request started
<code>waitlist_signup</code>	<code>source</code> , <code>country</code> , <code>duplicate</code>	The conversion
<code>waitlist_failed</code>	<code>source</code> , <code>status</code>	Server or network failure
<code>contact_message_sent</code>	-	No message body or address captured

`source` is already threaded through every waitlist CTA (`nav`, `nav-mobile`, `hero`, ...), so **"which placement converts"** is answerable with no extra instrumentation. That was the main reason to

instrument this funnel first.

Open decision: consent banner

There is **no cookie-consent banner**. The setup is deliberately light-touch (no person profiles, no replay, DNT honoured, first-party proxy), but under a strict GDPR reading, analytics cookies still want prior consent for EU visitors. Options: leave as-is, add a banner, or run cookieless (`persistence: "memory"`) until consent. **This is an unmade decision, not an oversight.**

9. Growth - what is actually available pre-launch

Most launch directories reject announced-only products. Verified rules:

Channel	Status	Rule
AlternativeTo	 Blocked	<i>"If the app is Coming Soon (announced only), in closed/private beta, invite-only, or in Early Access phase, we do not accept it."</i>
Show HN	 Blocked	<i>"If your work isn't ready for users to try out, please don't do a Show HN." and "Don't post landing pages or fundraisers."</i>
App Store listing	 At launch	The #1 most-cited source in this category (77 citations)
YouTube demo	 Available	#2 most-cited (74 citations) - the highest-leverage move available now
Reddit (r/macapps, r/homelab)	 Available	#3 most-cited (65 citations). Genuine participation, not a launch post
BetaList / Peerlist / Uned	 Available	Built specifically for unlaunched products
HN as a normal submission	 Available	Show HN is barred, but submitting a blog post as a regular link is fine
Email the waitlist	 Available	Never used yet
Product Hunt	Done	Badge is in the footer. Never solicit upvotes.

AlternativeTo timing note: an account must be **at least 7 days old** before it can submit. The account has been created, so the clock is running and a submission can go in on launch day. Prepare logo, screenshots and a demo video.

Competitor landscape

Screens, Jump Desktop, Termius, plus the generic remote-desktop tools. **Termius is only a partial competitor** - the honest overlap is narrow: *"run a terminal on your own Mac without SSH setup."* Chasing "termius alternative" broadly would attract multi-host-SSH searchers who bounce, which pollutes waitlist quality while pre-launch. The comparison post is deliberately titled *"Termius alternative? Only for one job."*

10. Backend

Waitlist - `POST /api/waitlist`

Swappable provider chosen automatically from env, or forced with `WAITLIST_PROVIDER` :

Provider	Env	Notes
Firebase <i>(recommended)</i>	<code>FIREBASE_PROJECT_ID</code> , <code>FIREBASE_CLIENT_EMAIL</code> , <code>FIREBASE_PRIVATE_KEY</code>	Writes to <code>waitlist</code> ; doc id = the normalized email , which is what de-dupes
Formspree	<code>FORMSPREE_ENDPOINT</code>	Zero-backend fallback
Resend	<code>RESEND_API_KEY</code> , <code>RESEND_AUDIENCE_ID</code>	409 is treated as duplicate
console <i>(default)</i>	-	Logs and succeeds, so the form works with no setup

Validation: email regex + ≤ 254 chars, name ≥ 2 chars, country must be in `lib/countries.ts` . Protections: hidden **honeypot** field (`website`) that returns a fake success to bots, and a **per-IP rate limit of 5/min**.

Contact - `POST /api/contact`

Same protections; writes to the Firestore `contacts` collection.

Transactional email - `lib/email.ts`

Sends the waitlist confirmation through **Resend's REST API over plain fetch** (no SDK - `waitlist-providers.ts` already talks to Resend the same way).

- **No-op without `RESEND_API_KEY`** . Signups still succeed, silently, in ~ 14 ms with no network attempt. Local dev and previews therefore never send real mail.
- **Never throws into the request path**. Every failure resolves. A signup that was already persisted must not be reported as failed because the email did not go out - the visitor would sign up again and get a duplicate response.
- **Awaited, not fire-and-forget**. Serverless functions can be frozen the moment the response returns, which silently drops an un-awaited send.
- **Not sent on duplicates**. That address already has the email.
- `List-Unsubscribe` uses a `mailto:` so it needs no endpoint to honour, which keeps the promise in the footer true from day one.

Sending domain: `hello@servey.in` , on Google Workspace. DNS at GoDaddy, and verified working end-to-end (SPF/DKIM/DMARC all PASS, DMARC at `p=quarantine`).

⚠ GoDaddy manages SPF through an `_spf` delegation (`v=spf1 include:dc-..._spf.servey.in ~all`), **not** a plain root record. Add Resend's include through GoDaddy's SPF interface. Hand-editing the root TXT into a second SPF record makes **both** fail and silently breaks all mail.

Deployment

Vercel, auto-deploy on push to `main`. Env vars go in **Settings → Environment Variables** for Production *and* Preview. `NEXT_PUBLIC_*` values are inlined at build time, so changing one requires a **redploy**, not just a restart.

11. Design system

Tokens are CSS custom properties on `:root` and `:root[data-theme="light"]` in `app/globals.css`, exposed to Tailwind v4 via `@theme inline: --bg --surface --border --border-strong --fg --muted --accent --accent-strong --accent-deep --accent-contrast --danger`, plus gradient stops `--sweep-a/b/c` and `--grad-a/b/c`.

Theme defaults to dark. An inline script in `<head>` reads `localStorage["servey-theme"]` before paint to avoid a flash.

Accessibility rules that were hard-won

- `--accent` is bright green and is NOT accessible on light backgrounds (1.72:1). Use `--accent-strong` for anything that must meet contrast - it resolves to `#22dc6e` in dark and `#0a7a3c` (5.2:1) in light. This has caused two separate regressions; check both themes.
- Gradient stops are **tokenized per theme**. They were once hardcoded near-white outside the light block, which made headline words invisible in light mode.
- **Framer Motion animates via the Web Animations API, so a CSS `prefers-reduced-motion` block cannot reach it.** Motion components must call `useReducedMotion()`. `RevealItem` passes `variants={reduce ? undefined : item}` rather than relying on inference that could pin sections at `opacity: 0`.
- Targets: 4.5:1 body text, 3:1 large text and focus indicators (SC 1.4.11).
- Heading order matters - `features.tsx` carries an `sr-only <h2>` because the page otherwise ran `h1 → h3×6 → h2` and the nav target had no accessible name.

Two CSS traps already hit

- Percentage `transform: translateX()` resolves against the element's own width, not the parent. `dual-path-diagram.tsx` measures the track with a `ResizeObserver` and animates in pixels for exactly this reason.
- `gap-px` grid dividers need a parent background to show through. The trust strip sets `bg-border` on the `<ul>` and `bg-bg` on the cells.

Screenshots - `lib/screenshots.ts`

Typed slots with `ratio`, `alt`, `src`, `ready`. `ready: false` renders the styled placeholder. `iphone-controls` and `dual-path` are still placeholders. **Ratios must match the real file** - two were wrong and `object-cover` was silently cropping.

12. Environment tooling note

The repo currently lives under `~/Desktop/...`, which macOS syncs to **iCloud Drive**. This has caused real, repeated failures: `bird / FPCKService` at 60–70% CPU, a 12.8-minute compile, killed type-checks, Turbopack PostCSS worker timeouts, a killed rebase and a killed merge.

It has also caused **data loss**: on 16 Aug 2026 `.git/HEAD` and four tracked files (`README.md` , `app/sitemap.ts` , `components/sections/features.tsx` , `public/llms.txt`) vanished from the working tree. Everything was recoverable from the object store, but the repo was briefly unusable.

Recommendation: move the repo off iCloud (e.g. `~/dev/servey-website`).

If git ever reports *"not a git repository"* while `.git` exists, check for a missing `.git/HEAD` first - restoring it with `ref: refs/heads/main` is usually the whole fix.

13. Outstanding items

Owner:

- Add `NEXT_PUBLIC_POSTHOG_KEY` in Vercel (Production + Preview), then redeploy
- Decide the consent-banner question (§8)
- Record the YouTube demo - highest-leverage channel available pre-launch
- Reddit participation in r/macapps and r/homelab
- ~~Email the waitlist~~ - **ruled out by the owner**; no bulk sends to waitlist members
- Request GSC indexing for the 7 newest URLs; check **Indexing → Pages** for "Discovered – currently not indexed"; open the generative-AI features report
- ~~Delete the 7 bogus Bing "sitemap" rows~~ - **done 1 Sep 2026** by the owner; Bing now shows 1 known sitemap, 0 warnings, 34 URLs discovered, crawled 1 Sep. Re-run the Ahrefs crawl
- Move the repo off iCloud (or exclude `.next`) - this is what causes the Turbopack PostCSS worker timeouts, not the config
- App Store + AlternativeTo submissions at launch
- **In-app account deletion - App Store submission blocker.** Guideline 5.1.1(v) has required it since 30 June 2022. There is no delete path in either app and `firestore.rules` sets `allow delete: if false` on `users`. Emailing `hello@servey.in` is honest and legally sufficient but is not what Apple checks for
- **Legal entity name, registered address and a named Grievance Officer** for the privacy policy - DPDP wants a person, not an `info@` alias
- **AccountView.swift:68** links `mailto:support@servey.in` while the site publishes `hello@servey.in`. Either create the alias or change the app - right now it is a dead contact route
- **Run the benchmark harness** (`scripts/benchmark/`) and fill the `<<FILL: ...>>` figures in `docs/benchmark-post-draft.md`. The draft is deliberately not in `lib/blog.ts`, so it cannot ship with invented numbers
- Capture the two missing screenshots: the **on-screen trackpad/keyboard** (feature 02 currently shows the Macs list - a knowing mismatch, commented in the slot) and the **iPad mirroring view** (`mirroring-ipad` is a placeholder)

Code, proposed but not approved:

- `LazyMotion` swap (~33 KB gzip off homepage First Load JS)
- Reduce eyebrow labels (10 against a budget of 3); break the run of 6 consecutive zigzag sections
- Re-shoot `hero-devices.png` (privacy - currently a deliberate "leave it")